

Homework 3: Latent Factors, Embeddings, and Attention

Math + Coding

Alexander Quispe Rojas
Universidad del Pacífico – MECA

Instructions. This homework covers Lectures 6 (latent factor models), 7 (embeddings), and 8 (attention). You may discuss the problems with classmates, but write up your own solutions. Show all work. Total: **120 points**.

Structure. (1) SVD & matrix factorization, (2) embeddings & word2vec, (3) attention & self-attention, (4) coding project (panel-data MF + self-attention from scratch).

Coding submission. Run `python _build_notebook.py` in `code/` to obtain `homework_3_student.ipynb`. Fill in the `TODO` cells. Submit the completed notebook alongside this PDF.

1 SVD and Matrix Factorization [30 Points]

Materials: Lecture 6 on latent factor models, SVD, PCA, matrix completion.

Problem 1 [4 Points]

Trace cyclic identity.

- (a) Prove $\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$ for $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times m}$.
- (b) Generalize: prove $\text{tr}(\mathbf{ABC}) = \text{tr}(\mathbf{BCA}) = \text{tr}(\mathbf{CAB})$ for conformable matrices.

Solution

(a) By definition,

$$\text{tr}(\mathbf{AB}) = \sum_{i=1}^m (\mathbf{AB})_{ii} = \sum_{i=1}^m \sum_{j=1}^n A_{ij} B_{ji}.$$

Swap the order of summation and the roles of the indices:

$$\sum_{i=1}^m \sum_{j=1}^n A_{ij} B_{ji} = \sum_{j=1}^n \sum_{i=1}^m B_{ji} A_{ij} = \sum_{j=1}^n (\mathbf{BA})_{jj} = \text{tr}(\mathbf{BA}).$$

(b) Treat $\mathbf{BC}$ as a single matrix. Applying (a) with $\mathbf{A}' = \mathbf{A}$ and $\mathbf{B}' = \mathbf{BC}$:

$$\text{tr}(\mathbf{A}(\mathbf{BC})) = \text{tr}((\mathbf{BC})\mathbf{A}) = \text{tr}(\mathbf{BCA}).$$

Applying (a) again with $\mathbf{A}' = \mathbf{B}$ and $\mathbf{B}' = \mathbf{CA}$:

$$\text{tr}(\mathbf{B}(\mathbf{CA})) = \text{tr}((\mathbf{CA})\mathbf{B}) = \text{tr}(\mathbf{CAB}). \quad \square$$

Homework 3: Latent Factors, Embeddings, and Attention

Problem 2 [4 Points]

SVD $\leftrightarrow$ PCA eigenvalues. Let $\mathbf{X} \in \mathbb{R}^{D \times N}$ have SVD $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ where $\mathbf{U} \in \mathbb{R}^{D \times D}$, $\mathbf{V} \in \mathbb{R}^{N \times N}$ are orthogonal and $\mathbf{\Sigma}$ is diagonal with singular values $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$.

- Show that $\mathbf{X}\mathbf{X}^\top = \mathbf{U}\mathbf{\Sigma}^2\mathbf{U}^\top$. Conclude that the eigenvalues of $\mathbf{X}\mathbf{X}^\top$ are exactly σ_d^2 .
- Use this to argue that the eigenvalues of $\mathbf{X}\mathbf{X}^\top$ are non-negative (no SVD required for the geometric reason).

Solution

(a) Substituting the SVD:

$$\mathbf{X}\mathbf{X}^\top = (\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top)(\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top)^\top = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top\mathbf{V}\mathbf{\Sigma}^\top\mathbf{U}^\top = \mathbf{U}\mathbf{\Sigma}\mathbf{\Sigma}^\top\mathbf{U}^\top = \mathbf{U}\mathbf{\Sigma}^2\mathbf{U}^\top,$$

using $\mathbf{V}^\top\mathbf{V} = \mathbf{I}$ and writing $\mathbf{\Sigma}\mathbf{\Sigma}^\top = \mathbf{\Sigma}^2$ (since $\mathbf{\Sigma}$ is diagonal with non-negative entries). Since $\mathbf{U}$ is orthogonal, this is exactly an eigendecomposition with eigenvalues σ_d^2 .

(b) For any non-zero $\mathbf{z} \in \mathbb{R}^D$,

$$\mathbf{z}^\top(\mathbf{X}\mathbf{X}^\top)\mathbf{z} = (\mathbf{X}^\top\mathbf{z})^\top(\mathbf{X}^\top\mathbf{z}) = \|\mathbf{X}^\top\mathbf{z}\|_2^2 \geq 0.$$

Hence $\mathbf{X}\mathbf{X}^\top$ is positive semidefinite, and all eigenvalues are non-negative. *Intuition:* the matrix encodes squared-magnitude information about projections of $\mathbf{X}$, which cannot be negative.

Problem 3 [3 Points]

Truncated SVD storage. Let $\mathbf{X} \in \mathbb{R}^{N \times N}$ have rank k . Suppose we store its rank- k SVD ($\mathbf{U}_k \in \mathbb{R}^{N \times k}$, $\mathbf{\Sigma}_k$, $\mathbf{V}_k^\top$) instead of the full matrix.

- How many real numbers do we need? (Hint: $\mathbf{\Sigma}_k$ is diagonal.)
- For what values of k is the truncated SVD strictly smaller than storing $\mathbf{X}$ as a dense $N \times N$ matrix?

Solution

(a) $\mathbf{U}_k$ has Nk entries, $\mathbf{V}_k$ has Nk entries, and $\mathbf{\Sigma}_k$ has k non-zero entries (the diagonal). Total: $2Nk + k = k(2N + 1)$.

(b) We need $k(2N + 1) < N^2$, i.e. $k < N^2/(2N + 1) \approx N/2$. So for $k \lesssim N/2$, the truncated SVD wins on storage; for $k \gtrsim N/2$, the dense matrix is cheaper.

Problem 4 [5 Points]

Matrix factorization gradients. Consider the regularized objective

$$L(\mathbf{U}, \mathbf{V}) = \frac{\lambda}{2}(\|\mathbf{U}\|_F^2 + \|\mathbf{V}\|_F^2) + \frac{1}{2} \sum_{(i,j) \in S} (y_{ij} - \mathbf{u}_i^\top \mathbf{v}_j)^2$$

where $\mathbf{U} \in \mathbb{R}^{M \times K}$, $\mathbf{V} \in \mathbb{R}^{N \times K}$, $\mathbf{u}_i^\top$ is the i -th row of $\mathbf{U}$, and S is the set of observed entries.

Derive $\partial L / \partial \mathbf{u}_i$ and $\partial L / \partial \mathbf{v}_j$ (treat these as column vectors in $\mathbb{R}^K$).

Solution

Only entries involving $\mathbf{u}_i$ contribute. The Frobenius norm gives $\partial(\frac{\lambda}{2}\|\mathbf{U}\|_F^2)/\partial\mathbf{u}_i = \lambda\mathbf{u}_i$. The reconstruction term:

$$\frac{\partial}{\partial\mathbf{u}_i} \frac{1}{2} \sum_{j:(i,j) \in S} (y_{ij} - \mathbf{u}_i^\top \mathbf{v}_j)^2 = - \sum_{j:(i,j) \in S} (y_{ij} - \mathbf{u}_i^\top \mathbf{v}_j) \mathbf{v}_j.$$

Combining,

$$\frac{\partial L}{\partial\mathbf{u}_i} = \lambda\mathbf{u}_i - \sum_{j:(i,j) \in S} (y_{ij} - \mathbf{u}_i^\top \mathbf{v}_j) \mathbf{v}_j$$

By symmetry,

$$\frac{\partial L}{\partial\mathbf{v}_j} = \lambda\mathbf{v}_j - \sum_{i:(i,j) \in S} (y_{ij} - \mathbf{u}_i^\top \mathbf{v}_j) \mathbf{u}_i$$

Problem 5 [5 Points]

ALS closed form. Fix $\mathbf{V}$ and solve for the $\mathbf{u}_i$ that minimizes L . Derive the closed-form update.

Solution

Set $\partial L/\partial\mathbf{u}_i = 0$:

$$\lambda\mathbf{u}_i + \sum_{j:(i,j) \in S} (\mathbf{u}_i^\top \mathbf{v}_j) \mathbf{v}_j = \sum_{j:(i,j) \in S} y_{ij} \mathbf{v}_j.$$

Using $(\mathbf{u}_i^\top \mathbf{v}_j) \mathbf{v}_j = \mathbf{v}_j \mathbf{v}_j^\top \mathbf{u}_i$:

$$\left(\lambda \mathbf{I}_K + \sum_{j:(i,j) \in S} \mathbf{v}_j \mathbf{v}_j^\top \right) \mathbf{u}_i = \sum_{j:(i,j) \in S} y_{ij} \mathbf{v}_j.$$

Therefore

$$\mathbf{u}_i = \left(\lambda \mathbf{I}_K + \sum_{j:(i,j) \in S} \mathbf{v}_j \mathbf{v}_j^\top \right)^{-1} \left(\sum_{j:(i,j) \in S} y_{ij} \mathbf{v}_j \right)$$

This is a regularized least-squares regression of user i 's observed ratings on the fixed item vectors. The matrix on the left is positive definite (sum of a PSD matrix and $\lambda \mathbf{I}$ with $\lambda > 0$), so the inverse always exists.

Problem 6 [9 Points]

Panel data with one missing potential outcome.

Econ application of matrix completion (Athey, Bai, Imbens, Khosravi, 2021).

Suppose we have a 4×3 outcome panel $\mathbf{Y}$ for 4 firms (rows) over 3 time periods (columns). Firm 3 in period 2 received treatment, so $Y_{3,2}$ is the *observed treated* outcome; we want the counterfactual $Y_{3,2}(0)$ to compute the treatment effect.

Homework 3: Latent Factors, Embeddings, and Attention

The data:

$$\mathbf{Y} = \begin{bmatrix} 2 & 4 & 6 \\ 1 & 2 & 3 \\ 3 & ? & 9 \\ 4 & 8 & 12 \end{bmatrix} \quad (? = \text{missing/treated}).$$

Assume rank $K = 1$ and use latent factor model $Y_{ij} \approx u_i \cdot v_j$ where $u_i, v_j \in \mathbb{R}$. **Suppose the column factors are fixed** at $\mathbf{v} = (1, 2, 3)$ (e.g., from a long pre-treatment period).

- Using the ALS update from Problem 5 with $\lambda = 0$, compute the optimal u_i for each row i given only the observed entries.
- Use $\hat{u}_3$ and the fixed $\mathbf{v}$ to predict the counterfactual $\hat{Y}_{3,2}(0)$.
- The observed treated outcome is $Y_{3,2}(1) = 8$. Report the estimated treatment effect $\hat{\tau}_{3,2} = Y_{3,2}(1) - \hat{Y}_{3,2}(0)$.
- Why does this approach generalize the synthetic control method? Answer in 2-3 sentences.

Solution

(a) With $\lambda = 0$ and $K = 1$, the ALS update for row i becomes

$$\hat{u}_i = \frac{\sum_{j \in S_i} y_{ij} v_j}{\sum_{j \in S_i} v_j^2}$$

where S_i is the set of observed time periods for row i .

For rows 1, 2, 4 we observe all 3 entries; for row 3 we observe only $j = 1$ and $j = 3$.

Row 1: $S_1 = \{1, 2, 3\}$. Numerator = $2(1) + 4(2) + 6(3) = 2 + 8 + 18 = 28$. Denominator = $1 + 4 + 9 = 14$.

$\hat{u}_1 = 28/14 = 2$.

Row 2: numerator = $1 + 4 + 9 = 14$, denom = 14. $\hat{u}_2 = 1$.

Row 3: $S_3 = \{1, 3\}$. Numerator = $3(1) + 9(3) = 3 + 27 = 30$. Denominator = $1 + 9 = 10$.

$\hat{u}_3 = 30/10 = 3$.

Row 4: numerator = $4 + 16 + 36 = 56$, denom = 14. $\hat{u}_4 = 4$.

(b) $\hat{Y}_{3,2}(0) = \hat{u}_3 \cdot v_2 = 3 \cdot 2 = 6$.

(c) $\hat{\tau}_{3,2} = 8 - 6 = \boxed{2}$.

(d) Synthetic control writes the treated unit's counterfactual as a convex combination of control units' outcomes. Matrix completion is more flexible: it imposes only a low-rank factor structure on the entire $\mathbf{Y}(0)$ matrix, allowing *multiple* treated units, *staggered adoption*, and no simplex restriction. Synthetic control is the special case of MC where we restrict to one treated unit with non-negative weights that sum to one.

2 Embeddings and Word2vec [30 Points]

Materials: Lecture 7 on embeddings, LLE, word2vec, hierarchical softmax, Huffman trees.

Problem 7 [5 Points]**Softmax gradient.**

The skip-gram conditional probability is

$$P(w_O | w_I) = \frac{\exp(\mathbf{v}'_{w_O} \mathbf{v}_{w_I})}{\sum_{w=1}^W \exp(\mathbf{v}'_w \mathbf{v}_{w_I})}$$

where $\mathbf{v}_w \in \mathbb{R}^D$ is the input embedding of word w and $\mathbf{v}'_w \in \mathbb{R}^D$ is its output embedding.

Derive $\nabla_{\mathbf{v}_{w_I}} \log P(w_O | w_I)$. Express the answer in terms of $\mathbf{v}'_{w_O}$, the model probabilities $P(w | w_I)$, and the output embeddings $\{\mathbf{v}'_w\}$. Give a one-sentence interpretation.

Solution

Write $Z(w_I) = \sum_w \exp(\mathbf{v}'_w \mathbf{v}_{w_I})$. Then

$$\log P(w_O | w_I) = \mathbf{v}'_{w_O} \mathbf{v}_{w_I} - \log Z(w_I).$$

First term: $\nabla_{\mathbf{v}_{w_I}} (\mathbf{v}'_{w_O} \mathbf{v}_{w_I}) = \mathbf{v}'_{w_O}$.

Second term, by chain rule:

$$\nabla_{\mathbf{v}_{w_I}} \log Z = \frac{1}{Z} \nabla_{\mathbf{v}_{w_I}} Z = \frac{1}{Z} \sum_w \exp(\mathbf{v}'_w \mathbf{v}_{w_I}) \mathbf{v}'_w = \sum_w P(w | w_I) \mathbf{v}'_w.$$

Combining:

$$\nabla_{\mathbf{v}_{w_I}} \log P(w_O | w_I) = \mathbf{v}'_{w_O} - \sum_{w=1}^W P(w | w_I) \mathbf{v}'_w$$

Interpretation: the gradient moves $\mathbf{v}_{w_I}$ toward the true context vector $\mathbf{v}'_{w_O}$ and away from the model's current expected context (a weighted average of all output vectors). This is the same “contrastive” structure that motivates negative sampling.

Problem 8 [3 Points]**Time complexity of full softmax.**

For one (w_I, w_O) pair, what is the time complexity of computing $\nabla_{\mathbf{v}_{w_I}} \log P(w_O | w_I)$ from scratch? Give the answer in terms of vocabulary size W and embedding dimension D , treating arithmetic operations as $O(1)$.

Solution

Each of the W exponentials in the denominator costs D multiplications to compute the dot product $\mathbf{v}'_w \mathbf{v}_{w_I}$, plus $O(1)$ for the exp. Computing $Z(w_I)$ is therefore $O(WD)$. The probabilities $P(w | w_I)$ are free once Z is known. Summing the weighted output vectors costs $O(WD)$ more. The first term is $O(D)$.

Total: $O(WD)$. *Note:* a naive implementation that recomputes Z inside the gradient sum would give $O(W^2D)$ – avoid this.

Problem 9 [9 Points]**Huffman tree vs balanced binary tree.**

Consider these 8 words with their frequencies from a small corpus:

Word	Count
<i>do</i>	18
<i>you</i>	4
<i>know</i>	7
<i>the</i>	20
<i>way</i>	9
<i>of</i>	4
<i>devil</i>	5
<i>queen</i>	6

Total: 73 occurrences.

- Build a Huffman tree for these words. Show your merging steps and report the resulting path length of each word. (Many valid trees; just produce *one*.)
- Build a balanced binary tree of depth 3 with all 8 words as leaves. What is the path length of each word?
- Compute the *expected* path length $\mathbb{E}[L] = \sum_w f_w L_w / \sum_w f_w$ under each tree. Which is shorter?
- In one sentence, explain why Huffman beats balanced for Zipfian (heavy-tailed) word distributions.

Solution

(a) Huffman merging steps (always combine the two smallest):

1. $\text{you}(4) + \text{of}(4) = 8$
2. $\text{devil}(5) + \text{queen}(6) = 11$
3. $\text{know}(7) + 8 = 15$
4. $\text{way}(9) + 11 = 20$
5. $15 + \text{do}(18) = 33$
6. $\text{the}(20) + 20 = 40$
7. $33 + 40 = 73$ (root)

Resulting path lengths (one valid set):

Word	f_w	L_w	Word	f_w	L_w
do	18	2	way	9	3
the	20	2	queen	6	4
know	7	3	devil	5	4
you	4	4	of	4	4

(b) Balanced depth-3 tree: every leaf has $L_w = 3$.

(c) **Expected path lengths.**

Huffman: $\sum f_w L_w = 18(2) + 20(2) + 7(3) + 9(3) + 4(4) + 4(4) + 5(4) + 6(4) = 36 + 40 + 21 + 27 + 16 + 16 + 20 + 24 = 200$.

$$\mathbb{E}[L]_{\text{Huffman}} = 200/73 \approx 2.74.$$

Balanced: $\sum f_w L_w = 73 \cdot 3 = 219$, so $\mathbb{E}[L]_{\text{balanced}} = 3.00$.

Huffman wins by about 9% on this small example.

(d) On Zipfian data, a few very frequent words account for a large fraction of occurrences. Huffman assigns them the shortest paths, dramatically reducing expected path length. On natural language ($W = 10^5$, Zipf exponent ~ 1), the savings over a balanced tree are typically 30–50%.

Problem 10 [5 Points]

Negative sampling gradient.

In negative sampling we replace the full softmax with a binary classification objective. For one observed pair (w_I, w_O) and k noise samples $\tilde{w}_1, \dots, \tilde{w}_k$, the loss is

$$\mathcal{L} = -\log \sigma(\mathbf{v}_{w_O}^\top \mathbf{v}_{w_I}) - \sum_{i=1}^k \log \sigma(-\mathbf{v}_{\tilde{w}_i}^\top \mathbf{v}_{w_I})$$

where $\sigma(z) = 1/(1 + e^{-z})$ is the sigmoid.

- Derive $\nabla_{\mathbf{v}_{w_I}} \mathcal{L}$.
- Compare your answer to the full-softmax gradient (Problem 7). What replaces the sum over all W output vectors?
- State the per-pair time complexity in terms of k and D .

Solution

(a) Use the identity $\frac{d}{dz}[-\log \sigma(z)] = \sigma(z) - 1$ and the chain rule.

For the positive term, $z = \mathbf{v}_{w_O}^\top \mathbf{v}_{w_I}$, so

$$\nabla_{\mathbf{v}_{w_I}} [-\log \sigma(z)] = (\sigma(z) - 1) \mathbf{v}_{w_O}'.$$

For each negative term, $z_i = -\mathbf{v}_{\tilde{w}_i}^\top \mathbf{v}_{w_I}$, with $\partial z_i / \partial \mathbf{v}_{w_I} = -\mathbf{v}_{\tilde{w}_i}'$:

$$\nabla_{\mathbf{v}_{w_I}} [-\log \sigma(z_i)] = (\sigma(z_i) - 1)(-\mathbf{v}_{\tilde{w}_i}') = (1 - \sigma(-\mathbf{v}_{\tilde{w}_i}^\top \mathbf{v}_{w_I})) \mathbf{v}_{\tilde{w}_i}' = \sigma(\mathbf{v}_{\tilde{w}_i}^\top \mathbf{v}_{w_I}) \mathbf{v}_{\tilde{w}_i}'.$$

(Using $1 - \sigma(-z) = \sigma(z)$.)

Combining:

$$\nabla_{\mathbf{v}_{w_I}} \mathcal{L} = (\sigma(\mathbf{v}_{w_O}^\top \mathbf{v}_{w_I}) - 1) \mathbf{v}_{w_O}' + \sum_{i=1}^k \sigma(\mathbf{v}_{\tilde{w}_i}^\top \mathbf{v}_{w_I}) \mathbf{v}_{\tilde{w}_i}'$$

(b) The full-softmax gradient has $\sum_w P(w | w_I) \mathbf{v}'_w$ (a weighted average over all W output vectors). Negative sampling replaces it with a sum over only k sampled negatives. Each negative is pushed away with weight $\sigma(\cdot) \in (0, 1)$ instead of $P(w | w_I)$.

(c) $O((k+1)D) = O(kD)$. With k typically 5–20, this is orders of magnitude cheaper than $O(WD)$ for $W \sim 10^5$.

Problem 11 [8 Points]

LLE numerical example.

Three collinear points in $\mathbb{R}^2$:

$$\mathbf{x}_1 = (0, 0), \quad \mathbf{x}_2 = (1, 1), \quad \mathbf{x}_3 = (2, 2).$$

Use $B = 2$ nearest neighbors throughout. Target dimension $K = 1$.

- Step 1.** For the interior point $\mathbf{x}_2$ with $\mathcal{B}(2) = \{1, 3\}$, find the weights $W_{2,1}, W_{2,3}$ subject to $W_{2,1} + W_{2,3} = 1$. Show your work.
- Step 2.** With $\mathbf{W}$ fixed, find low-dim coordinates $u_1, u_2, u_3 \in \mathbb{R}$ that satisfy the same convex-combination relation $u_2 = W_{2,1}u_1 + W_{2,3}u_3$, subject to (i) centering $\sum_i u_i = 0$ and (ii) unit variance $\frac{1}{3} \sum_i u_i^2 = 1$.
- Verify your answer recovers the 1D ordering and that the local reconstruction holds.

Solution

(a) Minimize $\|\mathbf{x}_2 - W_{2,1}\mathbf{x}_1 - W_{2,3}\mathbf{x}_3\|^2$ s.t. $W_{2,1} + W_{2,3} = 1$.
Substitute $\mathbf{x}_1 = 0$, $\mathbf{x}_2 = (1, 1)$, $\mathbf{x}_3 = (2, 2)$:

$$\mathbf{x}_2 - W_{2,1}(0, 0) - W_{2,3}(2, 2) = (1 - 2W_{2,3}, 1 - 2W_{2,3}).$$

Squared norm $= 2(1 - 2W_{2,3})^2$, minimized at $W_{2,3} = 0.5$. Hence $W_{2,1} = 0.5$.

(b) The constraint becomes $u_2 = 0.5u_1 + 0.5u_3$, i.e. u_2 is the midpoint.

Centering: $u_1 + u_2 + u_3 = 0$. Substituting $u_2 = 0.5(u_1 + u_3)$:

$$1.5(u_1 + u_3) = 0 \Rightarrow u_3 = -u_1, \quad u_2 = 0.$$

Unit variance: $\frac{1}{3}(u_1^2 + 0 + u_1^2) = 1 \Rightarrow u_1^2 = 1.5 \Rightarrow u_1 = -\sqrt{1.5}$ (choosing the negative root so the ordering matches the original x -direction; the positive root is equally valid – LLE is sign-invariant).

$$(u_1, u_2, u_3) = (-\sqrt{1.5}, 0, \sqrt{1.5}) \approx (-1.22, 0, 1.22)$$

(c) The ordering $u_1 < u_2 < u_3$ matches the original ordering along the line $y = x$. Local reconstruction check: $0.5 \cdot (-1.22) + 0.5 \cdot 1.22 = 0 = u_2$. ✓

3 Attention and Self-Attention [25 Points]

Materials: Lecture 8 on attention, scaled dot-product, multi-head attention, masking.

Problem 12 [4 Points]

Scaled dot-product attention: shapes.

Self-attention is given by

$$\text{Attention}(\mathbf{x}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

where $\mathbf{Q} = \mathbf{x}\mathbf{W}_Q^\top$, $\mathbf{K} = \mathbf{x}\mathbf{W}_K^\top$, $\mathbf{V} = \mathbf{x}\mathbf{W}_V^\top$.

The input $\mathbf{x} \in \mathbb{R}^{T \times D}$ has T tokens of embedding dim D . Key dim is d_k , value dim is d_v .

- Give the shapes of $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$.
- Give the shapes of $\mathbf{Q}, \mathbf{K}, \mathbf{V}$, and the score matrix $\mathbf{Q}\mathbf{K}^\top$.
- Give the shape of the final output.

Solution

- $\mathbf{W}_Q \in \mathbb{R}^{d_k \times D}$, $\mathbf{W}_K \in \mathbb{R}^{d_k \times D}$, $\mathbf{W}_V \in \mathbb{R}^{d_v \times D}$.
- $\mathbf{Q} \in \mathbb{R}^{T \times d_k}$, $\mathbf{K} \in \mathbb{R}^{T \times d_k}$, $\mathbf{V} \in \mathbb{R}^{T \times d_v}$. Score matrix $\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{T \times T}$.
- Output $\in \mathbb{R}^{T \times d_v}$.

Problem 13 [8 Points]

Why $\sqrt{d_k}$? A CLT argument.

Assume the entries of $\mathbf{q}, \mathbf{k} \in \mathbb{R}^{d_k}$ are i.i.d. $\mathcal{N}(0, 1)$ and that $\mathbf{q}$ and $\mathbf{k}$ are independent. Define the dot-product score $s = \mathbf{q}^\top \mathbf{k}$.

- Compute $\mathbb{E}[s]$.
- Compute $\text{Var}(s)$.
- Argue (via the CLT) that for large d_k , the distribution of s is approximately Gaussian. State its parameters.
- Why does dividing s by $\sqrt{d_k}$ “fix” the softmax-saturation problem? Be specific about what variance you obtain after scaling.

Solution

Write $s = \sum_{i=1}^{d_k} q_i k_i$.

- By independence, $\mathbb{E}[q_i k_i] = \mathbb{E}[q_i] \mathbb{E}[k_i] = 0 \cdot 0 = 0$. Sum of zeros: $\mathbb{E}[s] = 0$.
- Variance of each term: $\text{Var}(q_i k_i) = \mathbb{E}[(q_i k_i)^2] - 0 = \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] = 1 \cdot 1 = 1$. Independence

gives

$$\text{Var}(s) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = d_k.$$

(c) The d_k terms $q_i k_i$ are i.i.d. with mean 0 and variance 1, satisfying the CLT. Hence

$$s \approx \mathcal{N}(0, d_k).$$

Standard deviation: $\sqrt{d_k}$.

(d) For $d_k = 64$ (BERT-base), the score scale is $\pm\sqrt{d_k} = \pm 8$. Softmax inputs of magnitude ~ 8 produce nearly one-hot outputs (a logit gap of 8 means $e^8 \approx 3000\times$ weight on the largest). This saturates: $\nabla \text{softmax} \approx 0$.

After scaling:

$$\text{Var}\left(\frac{s}{\sqrt{d_k}}\right) = \frac{\text{Var}(s)}{d_k} = \frac{d_k}{d_k} = 1.$$

The scaled score is $\mathcal{N}(0, 1)$, keeping softmax in its sensitive (non-saturated) regime. Gradients flow.

Problem 14 [4 Points]

Multi-head attention with output projection.

In multi-head attention, h heads each compute

$$\mathbf{O}^i = \text{softmax}\left(\frac{\mathbf{Q}^i (\mathbf{K}^i)^\top}{\sqrt{d_k}}\right) \mathbf{V}^i$$

where $\mathbf{Q}^i = \mathbf{x} \mathbf{W}_Q^{i,\top}$, etc. The head outputs are concatenated and projected:

$$\mathbf{O} = [\mathbf{O}^1 \mathbf{O}^2 \dots \mathbf{O}^h] \mathbf{W}_O^\top.$$

The final output dimension is D .

Give the shapes of $\mathbf{W}_Q^i, \mathbf{W}_K^i, \mathbf{W}_V^i, \mathbf{W}_O$ in terms of D, d_k, d_v, h .

Solution

$$\mathbf{W}_Q^i, \mathbf{W}_K^i \in \mathbb{R}^{d_k \times D}, \quad \mathbf{W}_V^i \in \mathbb{R}^{d_v \times D}, \quad \mathbf{W}_O \in \mathbb{R}^{D \times h d_v}.$$

Common convention: $d_k = d_v = D/h$, so total parameter count across all heads roughly matches a single-head attention with full dim D .

Problem 15 [5 Points]

Numerical attention example.

Take $T = 2$, $d_k = d_v = 2$. Given:

$$\mathbf{Q} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix}.$$

Compute the attention output $\mathbf{O} = \text{softmax}(\mathbf{Q} \mathbf{K}^\top / \sqrt{d_k}) \mathbf{V}$ by hand, showing each step.

Solution**Step 1. $\mathbf{QK}^\top$:**

$$\mathbf{QK}^\top = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

Step 2. Divide by $\sqrt{d_k} = \sqrt{2} \approx 1.414$:

$$\frac{\mathbf{QK}^\top}{\sqrt{2}} \approx \begin{bmatrix} 0.707 & 0.707 \\ 0.707 & -0.707 \end{bmatrix}.$$

Step 3. Row-wise softmax.

Row 1: both entries are 0.707, so by symmetry softmax = (0.5, 0.5).

Row 2: entries are (0.707, -0.707). Difference = 1.414. Softmax = $\left(\frac{e^{0.707}}{e^{0.707} + e^{-0.707}}, \frac{e^{-0.707}}{e^{0.707} + e^{-0.707}} \right)$. Numerically: $e^{0.707} \approx 2.028$, $e^{-0.707} \approx 0.493$, sum ≈ 2.521 . So row 2 $\approx (0.804, 0.196)$.

Attention weights:

$$\mathbf{A} \approx \begin{bmatrix} 0.500 & 0.500 \\ 0.804 & 0.196 \end{bmatrix}.$$

Step 4. $\mathbf{O} = \mathbf{AV}$:

$$\mathbf{O} \approx \begin{bmatrix} 0.5 \cdot 2 + 0.5 \cdot 0 & 0.5 \cdot 0 + 0.5 \cdot 4 \\ 0.804 \cdot 2 + 0.196 \cdot 0 & 0.804 \cdot 0 + 0.196 \cdot 4 \end{bmatrix} = \begin{bmatrix} 1.000 & 2.000 \\ 1.608 & 0.784 \end{bmatrix}.$$

Reading: the first query attends equally to both values (symmetric scores); the second query is more selective and pulls strongly toward $\mathbf{v}_1 = (2, 0)$.

Problem 16 [4 Points]**Causal mask: explicit example.**

In a decoder language model, position i must not attend to positions $j > i$. The causal-masked score is

$$e_{i,j} = \begin{cases} \mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d_k}, & j \leq i, \\ -\infty, & j > i. \end{cases}$$

Suppose $T = 4$ and the unmasked scaled scores are

$$\frac{\mathbf{QK}^\top}{\sqrt{d_k}} = \begin{bmatrix} 0 & 1 & 2 & 3 \\ 1 & 0 & 1 & 2 \\ 2 & 1 & 0 & 1 \\ 3 & 2 & 1 & 0 \end{bmatrix}.$$

- Write the score matrix after applying the causal mask.
- For row $i = 2$ (zero-indexed: $i = 1$), write the resulting softmax row.

Homework 3: Latent Factors, Embeddings, and Attention

Solution

(a) Replace the upper-triangular entries above the diagonal with $-\infty$:

$$\mathbf{E}_{\text{masked}} = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 1 & 0 & -\infty & -\infty \\ 2 & 1 & 0 & -\infty \\ 3 & 2 & 1 & 0 \end{bmatrix}.$$

(b) Row 2 (i.e., the second row of $\mathbf{E}_{\text{masked}}$, indexing from 1) becomes $(1, 0, -\infty, -\infty)$. After softmax, $e^{-\infty} = 0$, so:

$$\text{row} = \left(\frac{e^1}{e^1 + e^0}, \frac{e^0}{e^1 + e^0}, 0, 0 \right) = \left(\frac{e}{e + 1}, \frac{1}{e + 1}, 0, 0 \right) \approx (0.731, 0.269, 0, 0).$$

Position 2 only attends to positions 1 and 2; the future positions 3 and 4 receive zero weight.
✓

4 Coding Project [35 Points]

Materials: Lectures 6, 7, and 8. See `code/` for the notebook generator and `code/data/` for the `dr_seuss.txt` corpus.

Problem 17 [10 Points]**Matrix factorization on synthetic panel data.**

Open `homework_3_student.ipynb`. The notebook generates a synthetic $M = 20$ firms $\times$ $T = 12$ months panel of (log) sales with a known low-rank structure plus noise, then masks 25% of the entries to simulate a staggered-adoption treatment.

You will:

- Implement matrix factorization via SGD using your gradients from Problem 4. Initialize $\mathbf{U}, \mathbf{V}$ uniformly in $[-0.5, 0.5]$. Use learning rate $\eta = 0.03$.
- Sweep $K \in \{1, 2, 3, 5, 8, 12\}$ and plot in-sample loss E_{in} and held-out (masked) loss E_{out} vs K .
- Identify the optimal K . How close is it to the true rank of the data-generating process?
- Repeat with regularization $\lambda \in \{0, 10^{-3}, 10^{-2}, 10^{-1}\}$. Briefly describe the bias-variance trade-off you observe.

Submit your completed notebook and a 4-line summary of part (d) here.

Solution

See `homework_3_solution.ipynb`. Typical findings:

- E_{in} decreases monotonically as K grows. E_{out} has a U-shape, minimized near the true rank (which is 3 in our generator).
- For $K >$ true rank, the model fits noise – E_{out} deteriorates.
- Regularization shifts the optimal K upward: with $\lambda = 0.1$, even $K = 12$ has reasonable E_{out} because the regularizer “soft-shrinks” unused dimensions.
- This is the matrix-completion analogue of bias-variance: K controls capacity, λ controls how aggressively that capacity is used.

Problem 18 [5 Points]**Self-attention from scratch.**

In the same notebook, you will implement `SelfAttention` and `MultiHeadAttention` as PyTorch modules (no `nn.MultiheadAttention`).

- Implement single-head scaled dot-product self-attention. Verify your output matches `torch.nn.functional.` to numerical tolerance ($< 10^{-5}$) on a random input.
- Extend to multi-head ($h = 4$) with an output projection $\mathbf{W}_O$. Confirm output shape is (T, D) matching the input.

Submit the completed cells in your notebook.

Solution

See the solution notebook. Key correctness checks: (1) score matrix shape is (T, T) , (2) softmax is applied *along the last axis* (each query normalized independently), (3) output shape matches input. The multi-head version splits the embedding dim across heads, computes attention independently, concatenates, and applies $\mathbf{W}_O$ – this reproduces the canonical Transformer attention block up to the residual/layernorm wrapping that comes in Lecture 9.

Problem 19 [10 Points]**Word2vec implementation on dr_seuss.txt.**

In the same notebook (Part 3), you will implement a working skip-gram model end-to-end and inspect the learned embeddings on Dr. Seuss text. The corpus is provided in `code/data/dr_seuss.txt` (2,071 tokens, ~ 308 unique words).

- Tokenize the corpus, build the vocabulary, and implement `generate_traindata(tokens, w2i, window_size=4)` that returns `(trainX, trainY)` as arrays of one-hot vectors – one training pair per (center, context) inside a sliding window of half-width $s = 4$.
- Build a 2-layer linear network $\text{Linear}(W \rightarrow D) \rightarrow \text{Linear}(D \rightarrow W)$ (no activation in the hidden layer – this is the textbook skip-gram architecture). Use cross-entropy loss and Adam ($\text{lr} = 0.01$) for 500 epochs. Set $D = 10$.
- Extract the hidden-layer weight matrix as the embedding (*shape conventions*: `nn.Linear.weight` is `(out, in)`; transpose to get one row per word). Compute pairwise cosine similarities and report the top 30 most-similar word pairs (deduplicated).
- Briefly answer in your write-up: (i) what are the dimensions of the hidden-layer and output-layer weight matrices? (ii) Do the resulting pairs make sense? (iii) Why do so many of the top pairs look like *rhymes*?

Solution

See the solution notebook (Part 3).

(c) Hidden layer weight: shape $(D, W) = (10, 308)$. Output layer weight: shape $(W, D) = (308, 10)$. (PyTorch stores both as `(out, in)`.)

Top similar pairs typically include many rhymes: `(yet, met)`, `(wump, hump)`, `(goat, boat)`, `(ear, fear)`, `(today, gone)`, ...

Why rhymes? Dr. Seuss writes in rhyming verse, so rhyming words appear in structurally similar positions of his sentences – nearly identical *contexts*. The distributional hypothesis (“you shall know a word by the company it keeps”) says nothing about meaning per se; it says words appearing in similar contexts will have similar embeddings. In this corpus that surfaces rhyme rather than semantics. *On a larger, more linguistically diverse corpus* like Wikipedia, similar pairs would be predominantly semantic (e.g., `king/queen`, `Paris/France`).

Problem 20 [10 Points]**Applied Transformer: sentiment + attention visualization.**

Homework 3: Latent Factors, Embeddings, and Attention

~~Up to now we built attention modules from scratch. In Part 4 of the notebook you will work with a real, pretrained Transformer: `distilbert-base-uncased-finetuned-sst-2-english` (HuggingFace).~~

The model is 6 layers, 12 heads per layer, hidden dim 768, fine-tuned on the SST-2 binary-sentiment task. First run downloads ~250 MB; everything runs on CPU in seconds.

- (a) Load the model and tokenizer. Run sentiment inference on five finance-flavored sentences (provided). Report the predicted label and confidence for each.
- (b) Pick one of the five sentences. Extract the attention tensor from `outputs.attentions` – shape (layers, batch, heads, T , T). Plot the attention map for layer 3, head 0 as a heatmap, with token strings on both axes.
- (c) In a markdown cell, answer: (i) was the model’s prediction consistent with a human judgement? (ii) Looking at the heatmap, are the attention patterns interpretable – do tokens attend to plausibly related tokens? (iii) Try a sentence with *ambiguous* sentiment (e.g., “*The hike was widely expected but the press conference was hawkish*”). Does the model still produce a confident label? What changes in the attention map?

Solution

See the solution notebook (Part 4).

The model assigns confident POSITIVE to sentences with words like *exceeded*, *record*, *aggressive buyback*, and confident NEGATIVE to sentences with *rose sharply*, *plummeted*, *de-anchored*.

Attention maps from middle layers (3–4) often show meaningful patterns: function words attending to nearby content words; the [CLS] token attending strongly to the most sentiment-bearing word in the sentence. Lower layers attend mostly locally; higher layers spread their attention more globally.

For ambiguous sentences, the model’s confidence drops (probability closer to 0.5) and attention maps become more diffuse – no single token dominates as a strong sentiment signal. This is the model’s way of expressing uncertainty, and it’s something you can read directly from the attention weights – a kind of interpretability that RNNs do not provide.

End of Homework 3. *120 points total: Section 1 (30) + Section 2 (30) + Section 3 (25) + Section 4 (35).*